

Introducción

En esta actividad se estará realizando un modelo de regresión logística mediante el uso de validación cruzada sobre un dataset de Obesidad. Este dataset es titulado "Estimation of obesity levels based on eating habits and physical condition" y contiene datos de niveles de obesidad de México, Perú y Colombia. Fue creada en 2019 mediante un estudio realizado por Fabio Mendoza y Alexis De la Hoz que recolectó datos mediante encuestas y posteriormente obtuvo más datos mediante la técnica SMOTE, una técnica que genera datos artificiales para balancear las clases. Este dataset contiene un total de 17 variables y 2,111 registros, entre las que se encuentran datos de las personas como edad, altura, sus hábitos, que pueden repercutir en la variable de salida, que es su nivel de obesidad. Lo que se pretende con este estudio es clasificar a las personas en dos grupos: ExcesoDePeso y NoExcesoDePeso.

La regresión logística nos permitirá clasificar a las personas en ambos grupos basándose en sus **características**, cuyo resultado se analizará con distintos umbrales propuestos mediante una curva ROC. El motivo de proponer distintos umbrales es observar cómo afectan la exactitud, sensibilidad y especificidad del modelo para elegir el más adecuado. Durante el proceso, se utilizará validación cruzada, la cual consiste en dividir el conjunto de entrenamiento en k grupos para entrenar y validar el modelo múltiples veces, asegurando que los resultados de exactitud sean robustos y no dependan de una sola partición de datos.

Finalmente, dado que la variable de salida original no es binaria, se realizará un proceso de binarización para adaptar el dataset a los requerimientos de la regresión logística. La variable de salida original del dataset, Obesity, es una variable cualitativa con siete categorías: Insufficient_Weight, Normal_Weight, Overweight_Level_I, Overweight_Level_II, Obesity_Type_I, Obesity_Type_II y Obesity_Type_III. Para poder aplicar regresión logística binaria, se construyó una nueva variable llamada EsObeso, donde se agruparon como Clase 1 las categorías que representan un riesgo clínico relevante: Sobre peso (Overweight Level I y II) y Obesidad (Type I, II y III); y como Clase 0 las categorías de Peso Normal e Insuficiente. Este criterio de separación se eligió porque desde una perspectiva de salud pública, tanto el sobre peso como la obesidad representan condiciones que incrementan el riesgo de enfermedades crónicas, por lo que tiene sentido agruparlas como una sola clase de riesgo. Respecto al balance de clases, la distribución resultante es de aproximadamente 73% ExcesoDePeso y 27% NoExcesoDePeso. Este desequilibrio es esperado, ya que cinco de las siete categorías originales fueron asignadas a la Clase 1, mientras que solo dos corresponden a la Clase 0, por lo tanto, el modelo deberá interpretarse considerando esta distribución, y se asegurará que dicha proporción se mantenga constante en las particiones de entrenamiento y prueba para garantizar una evaluación justa y robusta del desempeño.

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('Datos.csv')
print(df.shape)
df.head()

# Eliminar Height y Weight para evitar sesgo por IMC
df = df.drop(columns=['Height', 'Weight'])

# Binarizar variable de salida
obeso_labels = [
    'Obesity_Type_I', 'Obesity_Type_II', 'Obesity_Type_III',
    'Overweight_Level_I', 'Overweight_Level_II'
]
df['ExcesoDePeso'] = df['Obesity'].apply(lambda x: 1 if x in obeso_labels else 0)

# Balance de clases
conteo = df['ExcesoDePeso'].value_counts()
proporcion = df['ExcesoDePeso'].value_counts(normalize=True) * 100

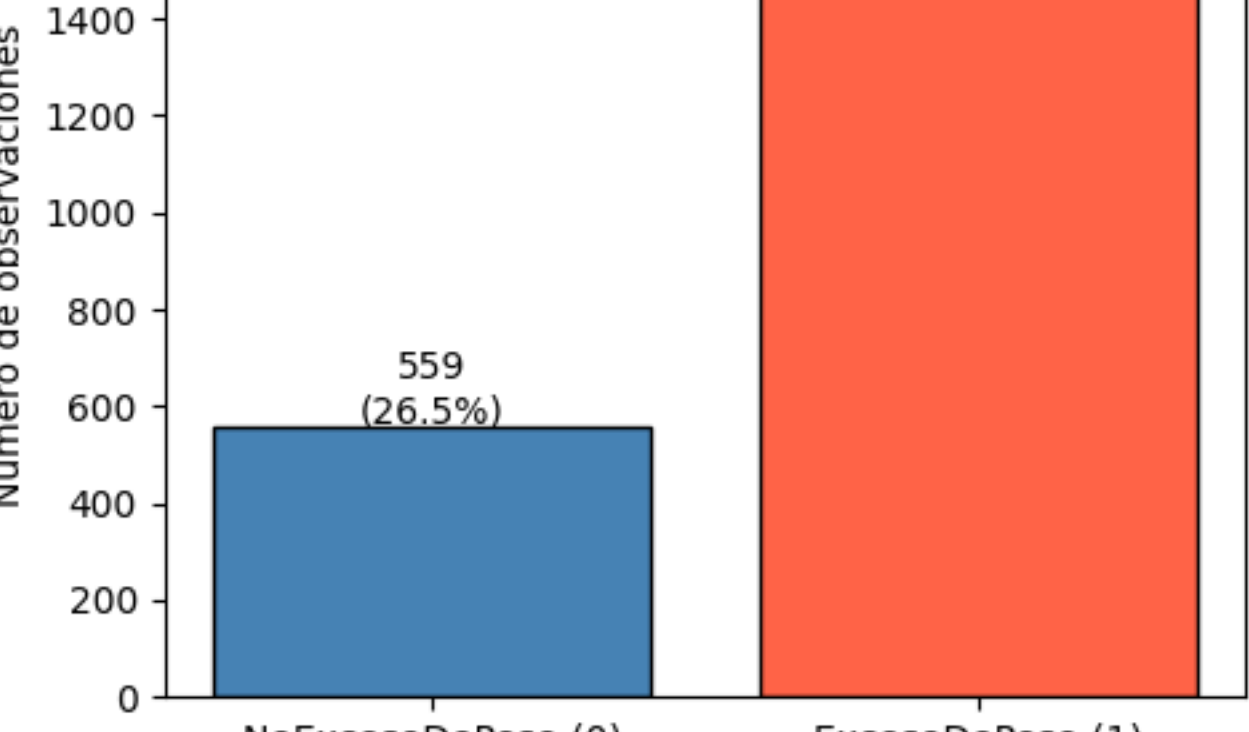
balance = pd.DataFrame({
    'Clase': ['NoExcesoDePeso (0)', 'ExcesoDePeso (1)'],
    'Conteo': [conteo[0], conteo[1]],
    'Porcentaje': [proporcion[0], proporcion[1]]
})

print('Balance de clases en la variable de salida ExcesoDePeso:')
print(balance.to_string(index=False))

fig, ax = plt.subplots(figsize=(5, 4))
ax.bar(['NoExcesoDePeso (0)', 'ExcesoDePeso (1)'], [conteo[0], conteo[1]],
       color=['steelblue', 'tomato'], edgecolor='black')
ax.set_ylabel('Número de observaciones')
ax.set_title('Balance de clases - Variable ExcesoDePeso')
for i, v in enumerate([conteo[0], conteo[1]]):
    ax.text(i, v + 10, f'({v/(proporcion[i]-1f)%}', ha='center', fontsize=10)
ax.set_ylim(0, max(conteo[0], conteo[1]) * 1.15)
plt.tight_layout()
plt.show()
```

(2111, 17)
Balance de clases en la variable de salida ExcesoDePeso:

	Clase	Conteo	Porcentaje
NoExcesoDePeso	(0)	559	26.480341
ExcesoDePeso	(1)	1552	73.519659



Metodología

Para la construcción del modelo, primero se realizará una limpieza y transformación de los datos. Se decidió eliminar las variables de altura y peso para evitar sesgo en el modelo, ya que el nivel de obesidad (variable objetivo) se calcula directamente mediante el IMC; al quitarlas, obligamos al modelo a encontrar patrones basados puramente en hábitos. Las variables categóricas de tipo string serán convertidas a numéricas mediante la técnica de One-Hot Encoding para poder aplicar la función logística. Respecto a la binarización de la variable de salida, se definirá como Clase 1 (ExcesoDePeso) a los registros con categorías de Obesidad y Sobre peso, y como Clase 0 (NoExcesoDePeso) a los de peso normal e insuficiente.

El conjunto de datos se dividirá en un 90% para entrenamiento y un 20% para prueba. Para garantizar que la proporción de clases se conserve en ambos subconjuntos, se utilizará una división estratificada mediante el parámetro stratify. Esto permitirá realmente que el modelo aprenda y sea robusto. Sin este parámetro el modelo podría obtener datos que fueran solo clase 1, y por lo tanto obteniendo una predicción errónea, por lo que este parámetro obliga a que el modelo escoga, según el número de registros con el que se vaya a entrenar, el mismo porcentaje que existe realmente en los datos originales, es decir 73% clase 1, y 27% clase 0.

Esta clasificación busca priorizar la sensibilidad del modelo, por lo que se experimentará con un umbral de decisión menor a 0.5, propuesto en 0.35. El objetivo de reducir el umbral es minimizar los falsos negativos, asegurando que las personas en riesgo de obesidad sean identificadas para tomar acciones preventivas, sacrificando un poco de especificidad en favor de una mayor cobertura de casos positivos. Adicionalmente, se evaluará un umbral mayor a 0.5, propuesto en 0.65, para observar el efecto contrario: mayor especificidad a costa de sensibilidad. Finalmente, la robustez de estos resultados se validará mediante Validación Cruzada K-Fold, donde se evaluará la exactitud promedio del modelo a través de múltiples particiones del conjunto de entrenamiento.

```
In [6]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# One-Hot Encoding
X_raw = df.drop(columns=['Obesity', 'ExcesoDePeso'])
X = pd.get_dummies(X_raw, drop_first=True)
y = df['ExcesoDePeso']

# Split 80/20 estratificado
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Escalar
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# Imprimir proporciones
def proporcion(y_series, nombre):
    p1 = (y_series.mean() * 100)
    p0 = (1 - y_series.mean()) * 100
    print(f'{nombre}: (n={len(y_series)}): ExcesoDePeoso={p1:.2f}% NoExcesoDePeso={p0:.2f}%')

proporcion(y, 'Datos originales ')
proporcion(y_train, 'Entrenamiento ')
proporcion(y_test, 'Prueba ')
```

Datos originales (n=2111): ExcesoDePeoso=73.52% NoExcesoDePeso=26.48%
Entrenamiento (n=1688): ExcesoDePeoso=73.52% NoExcesoDePeso=26.48%
Prueba (n=423): ExcesoDePeoso=73.52% NoExcesoDePeso=26.48%

Los resultados de la división muestran que la proporción de clases se conservó de forma exacta en los tres subconjuntos: tanto los datos originales, como los de entrenamiento y prueba presentan un 73.52% de observaciones de la Clase 1 (EsObeso) y un 26.48% de la Clase 0 (NoEsObeso). Esto se logró gracias al uso de la división estratificada con el parámetro stratify, que garantiza que cada subconjunto sea una muestra representativa del dataset original. Que las proporciones sean idénticas es importante porque asegura que el modelo se entrene y evalúe bajo las mismas condiciones de distribución, evitando que las métricas de desempeño se vean distorsionadas por una representación desigual de las clases en alguno de los subconjuntos.

Validación Cruzada

Para que el modelo sea realmente robusto, necesitamos realizar validación cruzada. La técnica que se estará utilizando de validación cruzada se llama Kfold cross-validation. Esta técnica consiste en separar en "k" grupos todos los registros y entrenar el modelo con todos los grupos menos uno, para después evaluar el grupo que se ocultó y observar el porcentaje de desempeño de los grupos que sirvieron de entrenamiento con respecto al grupo que fue oculto, y este ciclo se repite ocultando todos los grupos hasta que todos hayan sido probados. Al repetir el proceso "k" veces, se obtiene un promedio de desempeño que comparándolo con los demás procesos, indica si el modelo entiende que hábitos hacen que una persona este obesa o no. Este método se estará realizando con el 80% de datos que hemos designado para entrenamiento.

```
In [7]: from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score

modelo_cv = LogisticRegression(max_iter=1000, random_state=42)

skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_val_score(modelo_cv, X_train_sc, y_train, cv=skf, scoring='accuracy')

print('Exactitud por fold:')
for i, s in enumerate(scores, 1):
    print(f'Fold {i:2d}: {s*100:.2f}%')

print(f'\nExactitud promedio: {scores.mean()*100:.2f}%')
print(f'\nDesviación estándar: {scores.std()*100:.2f}%')
```

Exactitud por fold:

Fold 1: 86.98%
Fold 2: 88.76%
Fold 3: 88.76%
Fold 4: 87.57%
Fold 5: 89.35%
Fold 6: 88.76%
Fold 7: 89.94%
Fold 8: 92.31%
Fold 9: 82.74%
Fold 10: 93.45%

Exactitud promedio: 88.86%
Desviación estándar: 2.78%

Los resultados de la validación cruzada muestran que el modelo de regresión logística alcanzó una exactitud promedio de 88.86% con una desviación estándar de 2.78%. La exactitud por fold varió entre un mínimo de 82.74% (Fold 9) y un máximo de 93.45% (Fold 10), lo que indica que el modelo es consistentemente bueno a través de las distintas particiones, aunque con cierta variabilidad natural esperada. La desviación estándar relativamente baja confirma que el desempeño del modelo no depende de una partición específica de los datos, sino que generaliza bien de forma consistente.

La utilidad de la validación cruzada queda evidenciada precisamente en esta variabilidad: si se hubiera evaluado el modelo con una sola partición entrenamiento-prueba, el resultado podría haber sido tan optimista como 93.45% o tan pesimista como 82.74%, dependiendo de qué observaciones cayeran en cada subconjunto. Al promediar los resultados de 10 particiones distintas, se obtiene una estimación más confiable y representativa del desempeño esperado del modelo ante datos nuevos.

```
In [8]: from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay

# Entrenar modelo final
modelo = LogisticRegression(max_iter=1000, random_state=42)
modelo.fit(X_train_sc, y_train)

# Probabilidades para clase 1
y_prob = modelo.predict_proba(X_test_sc)[:, 1]

umbrales = [0.35, 0.50, 0.65]

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('Matrices de Confusión por Umbral', fontsize=14, fontweight='bold')

print(f'Umbral:>8} {'Exactitud:>10} {'Sensibilidad:>13} {'Especificidad:>14}')
print('-' * 50)

for ax, umbral in zip(axes, umbrales):
    y_pred = (y_prob >= umbral).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

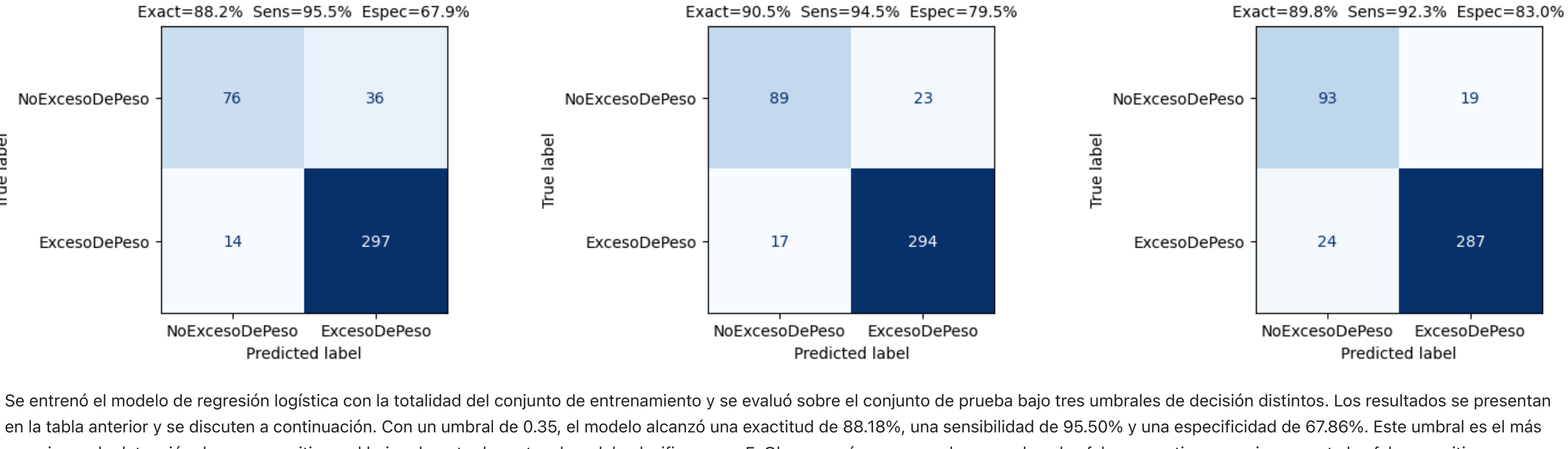
    exactitud = accuracy_score(y_test, y_pred)
    sensibilidad = tp / (tp + fn)
    especificidad = tn / (tn + fp)

    print(f'Umbral={umbral} {exactitud*100>9.2f}% {sensibilidad*100>12.2f}% {especificidad*100>13.2f}%')

    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['NoExcesoDePeso', 'ExcesoDePeso'])
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'Umbral = {umbral}\nExact={exactitud*100:.1f}% Sens={sensibilidad*100:.1f}% Espec={especificidad*100:.1f}%', fontsize=10)

plt.tight_layout()
plt.show()
```

Umbral	Exactitud	Sensibilidad	Especificidad
0.35	88.18%	95.50%	67.86%
0.50	90.54%	94.53%	79.46%
0.65	89.83%	92.28%	83.04%



Se entrenó el modelo de regresión logística con la totalidad del conjunto de entrenamiento y se evaluó sobre el conjunto de prueba bajo tres umbrales de decisión distintos. Los resultados se presentan en la tabla anterior y se discuten a continuación. Con un umbral de 0.35, el modelo alcanzó una exactitud de 88.18%, una sensibilidad de 95.50% y una especificidad de 67.86%. Este umbral es el más agresivo en la detección de casos positivos: al bajar el punto de corte, el modelo clasifica como EsObeso a más personas, lo que reduce los falsos negativos pero incrementa los falsos positivos, reflejándose en la especificidad más baja de los tres escenarios. En un contexto de salud pública, este umbral es el más recomendable, ya que es preferible identificar a todas las personas en riesgo aunque algunas clasificaciones sean incorrectas.

Con un umbral de 0.50, el modelo obtuvo su mayor exactitud con 90.54%, una sensibilidad de 94.53% y una especificidad de 79.46%. Este umbral representa el balance estándar entre ambas métricas y ofrece el mejor desempeño general en términos de exactitud. Con un umbral de 0.65, la exactitud fue de 89.83%, la sensibilidad bajó a 92.28% y la especificidad subió a 83.04%. Al elevar el umbral, el modelo se vuelve más conservador: solo clasifica como EsObeso a quienes tienen una probabilidad alta, lo que reduce los falsos positivos pero deja pasar más casos reales de obesidad sin detectar.

Comparando con los resultados de la validación cruzada, que reportó una exactitud promedio de 88.86% el modelo entrenado con todos los datos de entrenamiento y evaluado en el conjunto de prueba con umbral 0.50 alcanzó un 90.54%, ligeramente superior. Esta diferencia es pequeña y esperada: la validación cruzada estima el desempeño promedio sobre múltiples particiones, por lo que su resultado es más conservador y representativo del comportamiento general del modelo, mientras que el resultado sobre el conjunto de prueba puede variar dependiendo de qué observaciones lo conforman. La consistencia entre ambos valores confirma que el modelo generaliza bien y no presenta sobreajuste.

Análisis final

Para seguir verificando que tan robusto es nuestro modelo, realizamos la curva ROC y analizamos el Área bajo la Curva (AUC). Esta técnica consiste en observar la relación entre la tasa de verdaderos positivos y falsos positivos. Lo que se busca con esta gráfica es que el se obtenga una área bajo la curva cercana a 1, pues esto indica que el modelo realmente utiliza los hábitos de la gente (las variables) para identificar si una persona tiene o no exceso de peso.

```
In [10]: from sklearn.metrics import roc_curve, roc_auc_score

fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr, tpr, color='steelblue', lw=2, label=f'Modelo (AUC = {auc:.4f})')
ax.plot([0, 1], [0, 1], color='gray', linestyle='--', lw=1, label='Clasificador aleatorio')

# Marcar los tres umbrales
colores = ['green', 'orange', 'red']
for umbral, color in zip(umbrales, colores):
    idx = np.argmax(np.abs(thresholds - umbral))
    ax.scatter(fpr[idx], tpr[idx], color=color, zorder=5, s=80,
              label=f'Umbral={umbral} (FPR={fpr[idx]:.2f}, TPR={tpr[idx]:.2f})')

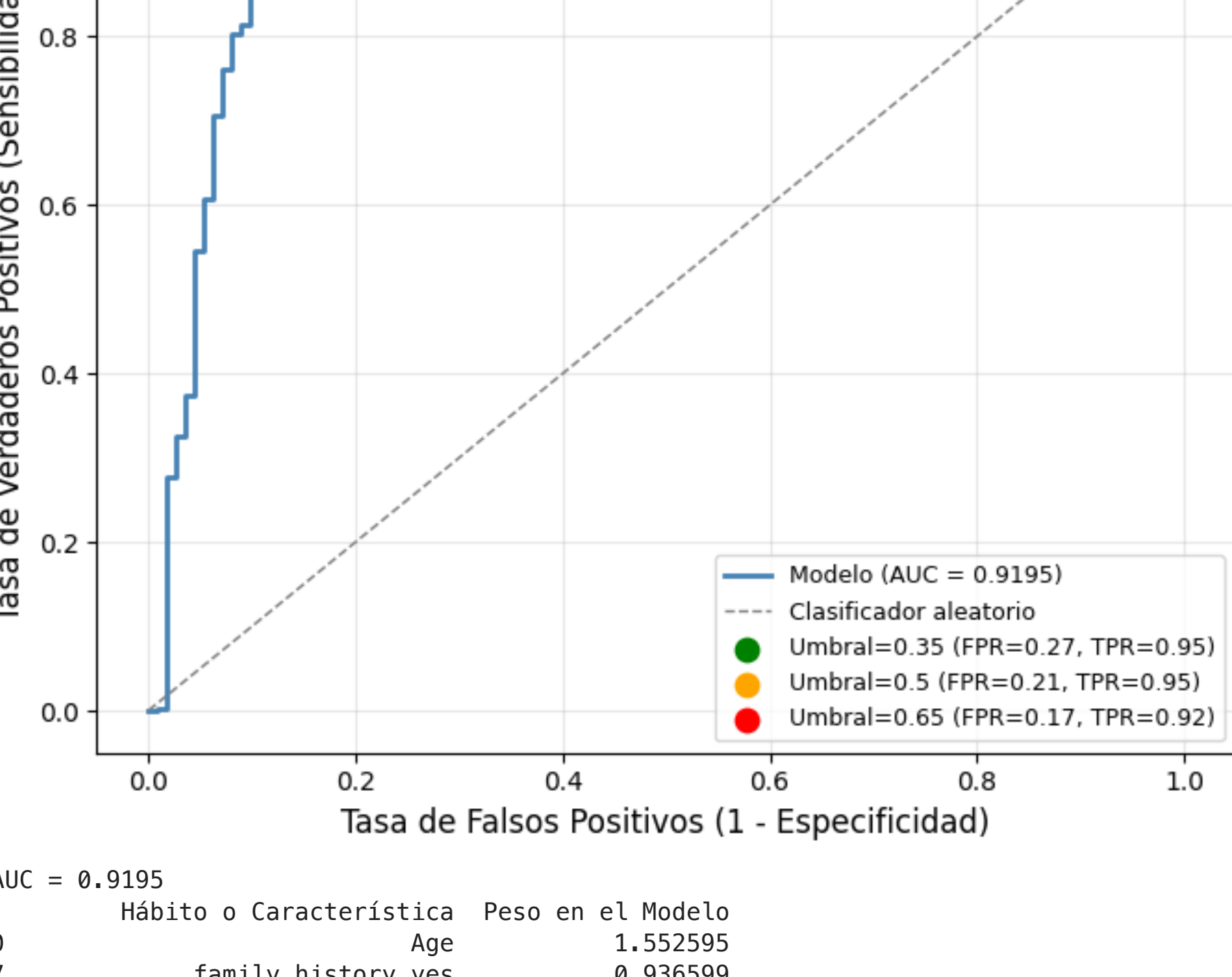
ax.set_xlabel('Tasa de Falsos Positivos (1 - Especificidad)', fontsize=12)
ax.set_ylabel('Tasa de Verdaderos Positivos (Sensibilidad)', fontsize=12)
ax.set_title('Curva ROC - Modelo de Regresión Logística', fontsize=13, fontweight='bold')
ax.legend(loc='lower right', fontsize=9)

ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f'AUC = {auc:.4f}')
```

importancias = pd.DataFrame({
 'Hábito o Característica': X.columns,
 'Peso en el Modelo': modelo.coef_[0]
})

Ordenamos de mayor a menor para ver qué influye más
importancias = importancias.sort_values(by='Peso en el Modelo', ascending=False)
print(importancias)



AUC = 0.9195

	Hábito o Característica	Peso en el Modelo
0	Age	1.552895
7	family_history_yes	0.936599
10	CAEC_Sometimes	0.850208
19	MTRANS_Public_Transportation	0.724080
11	CAEC_no	0.425117
8	FAVC_yes	0.162536
12	Gender_Male	0.162312
13	SCC_yes	0.155782
6	CH20	0.153722
14	CALC_Frequently	0.119782
1	FCVC	0.082322
18	MTRANS_Motorbike	0.055986
15	CALC_Sometimes	0.047524
17	MTRANS_Bike	-0.018986
20	MTRANS_Walking	-0.027916
16	CALC_no	-0.096387
12	SMOKE_yes	-0.102972
5	TUE	-0.180898
2	NCP	-0.244188
9	FAF	-0.267959
4	CAEC_Frequently	-0.384457

La curva ROC obtenida muestra el desempeño del modelo a través de todos los umbrales posibles, con un AUC de 0.9195. Este valor indica que el modelo tiene una excelente capacidad discriminativa: en el 91.95% de los casos, el modelo asigna una probabilidad mayor a una persona con obesidad que a una sin ella. La curva se aleja notablemente de la diagonal punteada que representa un clasificador aleatorio (AUC = 0.5), lo que confirma que las variables de hábitos y características personales son suficientes informativas para predecir la obesidad con alta precisión, incluso habiendo eliminado el peso y la altura. A parte, observamos los tres diferentes umbrales en la gráfica y esto nos permite saber que umbral es el mejor para nuestro modelo. Observamos que el umbral = 0.35 y el umbral = 0.5 tienen la misma tasa de verdaderos positivos, lo que significa que agitan mejor al clasificar personas, el umbral = 0.65 tiene una tasa de verdaderos positivos un poquito más baja, lo que descarta a este umbral como el indicado para nuestro modelo. Para saber cual es el mejor, dado que 0.35 y 0.5 tienen la misma tasa de verdaderos positivos, observamos tenemos que observar la tasa de falsos positivos, donde la menor tasa es la del umbral = 0.5. Esto nos permite entender que este umbral es el mejor para clasificar los registros.

Finalmente, observamos los diferentes pesos que tienen los hábitos o características de las personas en el modelo y así entendemos cuáles influyen más. Observamos que la edad es la característica más importante, podemos inferir que la edad repercute en si una persona es obesa o no debido a que los hábitos cambian, el metabolismo no es el mismo... También observamos que el historial familiar es muy importante, con un peso de 0.936, esta característica reafirma el famoso dicho "de con quien te juntas y te diré quien eres", representando que los hábitos que tienen las personas de la familia repercuten en el estado físico de las personas. Los últimos dos hábitos que se observan que tienen bastante peso son si la persona consume alimentos entre comidas como snacks, y el método de transporte.

Referencias

Anthropic. (2024). Claude 3.5 Sonnet. <https://www.anthropic.com>

Codificando Bits. (2021, 15 de febrero). Validación cruzada (K-Fold Cross-Validation): qué es y cómo funciona. <https://codificandobits.com/blog/validacion-cruzada-k-fold-cross-validation/>

Google. (2024). Gemini 1.5 Flash. <https://gemini.google.com>

Palechor, F. M., & de la Hoz Manotas, A. (2019). Obesity Levels Based On Eating Habits and Physical Condition. UCI Machine Learning Repository. <https://archive.ics.uci.edu/ml/datasets/Estimation+of+obesity+levels+based+on+eating+habits+and+physical+condition+>